

PA1, segmentação de instâncias com as arquiteturas da aula

Documento de apoio da apresentação. Cada número aqui aponta pro arquivo em [results/](#) de onde ele saiu, e todo arquivo em [results/](#) sai de um comando que está no README.

O que o PA pede e o que a gente entendeu dele

A aula de segmentação semântica gastou o slide 4 fazendo a distinção entre rótulo class-aware e rótulo instance-aware, e depois passou oitenta slides tratando só do primeiro caso. O PA é a outra metade: fazer as mesmas arquiteturas (encoder-decoder, FCN, SegNet, U-Net, ResUNet, DeepLab, PSPNet) produzirem rótulo instance-aware sem detector com proposta de região.

O ponto que demorou pra gente entender é que não se trata de trocar de arquitetura. A mesma U-Net resolve os dois casos. O que muda são três coisas acopladas, e as três são decisão de projeto nossa:

1. O que a rede prevê, ou seja, a representação de saída
2. Qual perda otimiza aquilo
3. Como a previsão vira objeto, ou seja, o pós-processamento

A tese que a gente defende é que a decisão 1 (o que a rede prevê) é a que manda, e que a 2 (a perda) é ajuste fino em cima. A evidência principal é o experimento de oráculo mais adiante, que dá pra rodar antes de treinar qualquer rede.

Uma ressalva que a gente só descobriu medindo, e que vale dizer logo: no dataset sintético o gargalo é 100% a representação, mas no DSB2018 real a rede também limita. O modelo da Parte 1 tira 0.535 de mAP contra um teto de 0.766 do próprio decodificador dele. Então no real as duas coisas estão apertando ao mesmo tempo, e a gente escreveu a análise nesse sentido em vez de forçar a tese mais simples.

Dataset e split

Opção A, DSB2018 / BBBC038v1. São 670 imagens de treino, cada núcleo num PNG separado, sem sobreposição. Escolhemos ela porque baixa direto do Broad sem conta, não tem COCO pra parsear e, principalmente, porque núcleo encostado é a regra e não a exceção, que é exatamente o problema que o PA quer atacado.

O enunciado exige split estratificado por modalidade. O DSB2018 não traz essa coluna, então a gente construiu um proxy: descreve cada imagem com oito estatísticas de cor (média e desvio do cinza, saturação média, mediana do cinza, fração de pixel muito escuro, fração de pixel muito claro, e as diferenças R-G e B-G) e roda um k-means com 4 clusters. Depois o split é feito dentro de cada cluster. Reprodz com `uv run python scripts/split_report.py`, e o resultado está em [results/split_report.json](#).

Os clusters saem interpretáveis, o que é o que dá confiança de que o proxy funciona:

cluster	n	cinza médio	saturação	frac escuro	frac claro	núcleos/img	diâm mediano	leitura
0	449	0.041	0.000	0.992	0.000	24.1	22.8 px	fluorescência, fundo preto
1	70	0.647	0.350	0.000	0.005	36.8	17.4 px	histologia corada, roxo/rosa
2	54	0.771	0.110	0.001	0.720	63.5	17.9 px	brightfield, fundo claro
3	97	0.103	0.000	0.906	0.006	81.4	18.3 px	fluorescência densa

Duas observações que valem na apresentação. Primeira, o k-means separou modalidade e também regime de densidade: os clusters 0 e 3 são os dois fluorescência em escala de cinza, mas um tem 24 núcleos por imagem e o outro tem 81. Isso importa porque a Parte 1 pede exatamente a relação entre desempenho e densidade, e sem estratificar o cluster 3 poderia cair quase todo de um lado do split. Segunda, o cluster 0 sozinho é 67% do dataset, então um split aleatório provavelmente ficaria razoável por sorte, mas os clusters 1 e 2 têm 70 e 54 imagens e são os que corriam risco de verdade.

O split resultante é 469 treino, 100 validação, 101 teste. A maior diferença de proporção de cluster entre treino e teste ficou em **0.006**, ou seja, a estratificação funcionou.

cluster	treino	val	teste	% treino	% teste
0	314	67	68	0.670	0.673
1	49	10	11	0.104	0.109
2	38	8	8	0.081	0.079
3	68	15	14	0.145	0.139

A métrica de instância, e por que a gente teve que definir ela

O slide 6 define AP como precisão média sobre as classes. Isso é a métrica semântica e não serve aqui: ela não tem noção de objeto individual. O PA manda generalizar pro nível de instância e proíbe usar AP de biblioteca, então o matching é escrito por nós em <src/pa1/metrics/instance.py>.

A definição que a gente adotou, que é a convenção do próprio Data Science Bowl 2018: para cada limiar de IoU t em $\{0.50, 0.55, \dots, 0.95\}$, conta cada instância prevista com no máximo uma instância verdadeira. Par com IoU maior ou igual a t é TP, previsão sem par é FP, verdade sem par é FN, e a precisão naquele limiar é

$$P(t) = TP / (TP + FP + FN)$$

O AP da imagem é a média de $P(t)$ sobre os dez limiares, e o mAP do conjunto é a média dos AP das imagens. Vale registrar que esse denominador não é o de precisão no sentido usual (que seria $TP / (TP + FP)$): ele

penaliza FN junto, o que na prática torna a métrica bem mais dura e é o motivo de os números absolutos parecerem baixos.

A regra de matching é greedy por IoU decrescente, que é o item 4 da Parte 1. Ordena todos os pares candidatos com IoU maior ou igual ao limiar por IoU decrescente e vai casando, pulando quem já foi usado. A alternativa está implementada também (`--matching hungarian`, que resolve a atribuição global com `linear_sum_assignment` e só depois corta os pares abaixo do limiar) e a apresentação mostra as duas lado a lado. Guloso é mais duro em casos ambíguos porque uma escolha localmente boa pode roubar o par de outra instância, mas em quase toda imagem real os dois coincidem, porque um núcleo bem previsto tem IoU alto com um único núcleo verdadeiro e próximo de zero com todos os outros.

A implementação da matriz de IoU é vetorizada. Em vez de dois laços sobre instâncias, a gente codifica cada par de labels em `pred * (n_gt + 1) + gt` nos pixels onde os dois têm rótulo, conta com `bincount` e reformata pra matriz. Os testes em `tests/test_instance_metrics.py` conferem a conta na mão: match perfeito dá AP 1, um FN dá 0.5, um FP espúrio dá 2/3, um caso construído de IoU 8/16 bate, e dois objetos fundidos num blob só dão menos de 0.5.

Parte 0, o teste unitário sintético

O gerador está em `src/pa1/data/synthetic.py`. Imagens 128x128, de 5 a 20 elipses, raio de 5 a 18 px, ângulo e razão de eixos aleatórios, intensidade por instância, nível de fundo, borramento e ruído variáveis. Não guarda nada em disco: cada índice deriva uma seed, então o dataset é determinístico e os splits ficam disjuntos só mudando a seed base.

O detalhe que importa é que **as elipses têm que encostar**. A primeira versão sorteava posição uniforme e quase nenhuma encostava, o que tornava o teste inútil (componentes conexos resolveria tudo e a Parte 1 não teria fracasso nenhum pra mostrar). A versão final ancora 70% das elipses novas ao lado de uma já colocada, com a distância entre centros em torno da soma dos raios médios.

Dois bugs nossos apareceram nesse caminho e valem ser contados. O primeiro: elipse desenhada por cima enterra a anterior, e a imagem podia acabar com menos de 5 instâncias, fora da faixa que o PA pede. Arrumamos contando quantas instâncias sobrevivem a cada passo em vez de quantas foram desenhadas. O segundo só apareceu quando fomos medir o teto de oráculo, e está descrito na seção seguinte.

O experimento de oráculo, que é o argumento central do trabalho

Antes de treinar qualquer rede, dá pra responder a pergunta "o problema está na rede ou na representação?" alimentando o pós-processamento com a saída perfeita. Se a rede acertasse tudo, quanto cada decodificação entregaria?

Isso é `scripts/oracle_ceiling.py`, que não treina nada e não carrega checkpoint. Rodamos nos dois datasets, no split de teste inteiro. Saída em `results/oracle_ceiling_synthetic_test.json` e `results/oracle_ceiling_dsb2018_test.json`.

decodificação	entrada perfeita	sintético (128 img)	DSB2018 (101 img)
limiar + componentes conexos (Parte 1)	máscara semântica	0.115	0.766

decodificação	entrada perfeita	sintético (128 img)	DSB2018 (101 img)
watershed com marcadores (Parte 2)	fronteira e distância	0.791	0.960
watershed só do mapa de distância	só a distância	0.790	0.970

E só no terço mais denso de cada split, que é onde o problema aparece:

decodificação	sintético (≥ 16 obj)	DSB2018 (≥ 45 obj)
componentes conexos	0.077	0.660
watershed	0.739	0.954

Três leituras, e a terceira é a que mais vale falar na apresentação.

Primeira, **o teto do componentes conexos é um teto de verdade**. Com a máscara semântica perfeita, ou seja, com uma rede que não erra um pixel sequer, o método da Parte 1 não passa de 0.115 no sintético e 0.766 no DSB2018. Nenhum treino, nenhum encoder, nenhuma perda melhora isso, porque a informação que separa dois núcleos encostados não existe numa máscara binária: eles formam um blob único e conexo.

Segunda, **trocar o que a rede prevê muda o teto, com a mesma arquitetura**. De 0.115 para 0.791 no sintético e de 0.766 para 0.960 no DSB2018. E no terço mais denso, que é o caso que interessa, de 0.077 para 0.739 e de 0.660 para 0.954.

Terceira, e essa a gente só percebeu depois de rodar: **o dataset sintético é muito mais duro que o real**, de propósito. No sintético quase toda elipse encosta em outra, então componentes conexos é catastrófico (0.115). No DSB2018 boa parte dos núcleos está isolada, e componentes conexos já entrega 0.766. Ou seja, o sintético não é uma versão fácil do problema real, é uma versão concentrada exatamente no modo de falha que a gente quer atacar. Isso vai importar de novo na Parte 2, quando o ganho no real for menor que no sintético: não é o método funcionando pior, é o problema aparecendo em menor proporção.

O mesmo experimento em 24 imagens sintéticas está como teste em

`tests/test_targets_and_postprocess.py`

(`test_watershed_beats_connected_components_with_perfect_input`), então ele roda no `pytest` e trava se alguém quebrar a geração de alvo. Nessas 24, em 24 de 24 o componentes conexos funde pelo menos duas elipses.

O bug número dois apareceu aqui. Na primeira versão do gerador o teto do watershed dava 0.74 em vez de 0.83 nas 24 imagens de teste, e a gente foi investigar imagem por imagem esperando achar um erro no watershed. O culpado eram instâncias de 6 a 8 pixels, restos de elipses quase totalmente enterradas por outra desenhada por cima. Lasca de 6 pixels não é objeto, ninguém casa com ela, e ela puxava a métrica inteira pra baixo. Colocamos um filtro de área mínima no gerador e o teto subiu. Vale contar porque é um caso de a métrica estar certa e o **dataset** estar errado, e a gente quase foi mexer no lugar errado.

O teste unitário propriamente dito

O PA pede que treine em menos de 5 minutos e que a métrica seja reportada. Medimos nos dois lugares onde o código rodou.

Na GPU (Tesla P100 do Kaggle), com o config do repo (512 imagens de treino, 12 épocas), as duas versões:

config	tempo de treino	Dice (teste)	mAP (teste)	erro de contagem
synthetic_baseline (binário, Parte 1)	1.2 min	0.9920	0.1032	9.70
synthetic_boundary (trilha A, Parte 2)	1.3 min	0.9857	0.5122	2.22

Na CPU (i7 de 18 núcleos, sem GPU), com 320 imagens e 8 épocas, a versão binária leva 7.4 minutos no total, mas a métrica satura na quinta época, aos **4.8 minutos**:

época	minutos	loss	IoU	Dice	mAP
1	1.0	0.847	0.722	0.798	0.046
2	1.9	0.656	0.951	0.974	0.082
3	2.9	0.576	0.886	0.924	0.089
4	3.8	0.513	0.946	0.971	0.093
5	4.8	0.465	0.976	0.988	0.102
6	5.7	0.435	0.977	0.989	0.110
8	7.4	0.409	0.976	0.988	0.100

Dos dois lados o número pra levar é o mesmo: **Dice em torno de 0.99 e mAP de instância em torno de 0.10**. A rede resolveu a segmentação semântica de forma essencialmente perfeita, e o mAP de instância ficou igual ao teto de oráculo do componentes conexos (0.102), medido antes de treinar. Ou seja, o erro que sobra não é da rede, é inteiramente do decodificador. Isso justifica a decisão de selecionar checkpoint por mAP de validação e não por loss: aqui as duas coisas estão completamente descorreladas.

E a linha de baixo da primeira tabela já antecipa a Parte 2 no mesmo dataset e com a mesma arquitetura: trocando só o que a rede prevê, o mAP vai de **0.103 para 0.512** e o erro de contagem cai de **9.70 para 2.22 núcleos por imagem**. Quase 5 vezes de ganho, com Dice praticamente igual (0.9920 contra 0.9857, ou seja, ligeiramente pior).

Parte 1, o baseline e a quantificação do fracasso

configs/dsb2018_baseline.yaml: U-Net com encoder ResNet34 pré-treinado na ImageNet, decoder nosso com skip connections (slides 25 a 29), saída de 1 canal, perda BCE mais Dice, crop de 256, 40 épocas, AdamW com cosine annealing. Treino levou 5.4 minutos na P100. Instâncias saem por limiar 0.5 mais componentes conexos com conectividade 8 e descarte de blobs abaixo de 20 px.

Resultado no split de teste (101 imagens), em **results/parte1/metrics.json**:

métrica	valor
IoU semântico	0.8415

métrica	valor
Dice	0.9120
mAP @[.50:.95]	0.4654
AP @.50	0.6931
erro absoluto de contagem	10.05 núcleos por imagem

(esses números são com a decodificação padrão, limiar 0.5. Calibrando o limiar na validação, como a gente faz na Parte 2 pros dois modelos, o baseline sobe pra mAP 0.5351 e erro de contagem 8.57. A comparação justa está na Parte 2.)

Por limiar de IoU, que é o que mostra onde a coisa desmonta:

limiar	0.50	0.55	0.60	0.65	0.70	0.75	0.80	0.85	0.90	0.95
precisão	0.693	0.663	0.633	0.601	0.572	0.509	0.423	0.314	0.188	0.058

Quanto desse erro é da rede e quanto é do decodificador

Comparando com o teto de oráculo medido antes: o decodificador ingênuo, alimentado com a máscara perfeita, daria 0.766 nesse mesmo split. Esse mesmo modelo, com o limiar calibrado na validação (ver a Parte 2), entrega 0.535. Então o erro se reparte quase ao meio:

de onde vem	quanto	por que
culpa da rede	$0.766 - 0.535 = \mathbf{0.231}$	a máscara semântica não é perfeita
culpa do decodificador	$1.000 - 0.766 = \mathbf{0.234}$	núcleo encostado que nem máscara perfeita separa

Isso é diferente do que acontece no sintético, onde a rede praticamente satura o teto (0.103 medido contra 0.115 de teto do componentes conexos) e quase todo o erro é do decodificador. É uma nuance que a gente só viu depois de medir os dois, e ela importa pra Parte 2: no DSB2018 a mudança de representação ataca metade do problema, não o problema todo, e por isso o ganho aqui é menor que no sintético. Não é o método funcionando pior, é ele atacando uma fração menor do erro.

O número que mais importa aqui é o erro de contagem: **10 núcleos de erro por imagem**, num dataset onde a mediana é algo em torno de 25 núcleos por imagem. Com Dice de 0.91, ou seja, com a máscara semântica essencialmente certa, o método erra a contagem em cerca de 40%. É o mesmo padrão do sintético, só que menos extremo porque nem todo núcleo do DSB2018 encosta em outro.

A regra de matching, na prática

Rodamos a avaliação com as duas regras no mesmo checkpoint e no mesmo split:

regra	mAP	AP50	erro de contagem
guloso por IoU decrescente	0.4654	0.6931	10.05
Hungarian	0.4654	0.6931	10.05

Deram **exatamente o mesmo número**, até a quarta casa. Isso confirma o argumento que a gente ia fazer no papel: as duas regras só divergem quando existe ambiguidade real, e núcleo previsto razoavelmente bem tem IoU alto com um único núcleo verdadeiro e perto de zero com todos os outros, então a atribuição ótima e a gulosa coincidem. Reportamos as duas justamente porque o enunciado pede a regra explícita, e achamos honesto mostrar que nesse dataset a escolha não muda nada. Em um dataset com objetos muito sobrepostos a conclusão poderia ser outra.

Parte 2, trilha A: fronteira e watershed

Escolhemos a trilha A. As três razões, em ordem de peso:

O pós-processamento é determinístico. A trilha B (embeddings discriminativos) precisa de clustering na inferência, e mean-shift e DBSCAN trazem hiperparâmetro (largura de banda, eps, min_samples) que muda o número de objetos encontrados. Isso vira uma segunda fonte de erro que a gente teria que separar da qualidade da rede, e com dois alunos e prazo de duas semanas era risco demais.

A trilha C (centro mais offsets) supõe implicitamente que o objeto é mais ou menos convexo e que apontar pro centro identifica o objeto. Núcleo em divisão no DSB2018 é frequentemente bilobado, e nesses o centro geométrico às vezes cai fora da máscara.

E a trilha A é a que mais se encaixa nas peças que a aula deu: a perda de classe é CE balanceada ou focal (slides 73 a 79) e a de distância é L1 ou L2 (slide 80), tudo material de aula.

O que a rede prevê

O encoder-decoder é literalmente o mesmo da Parte 1, só a head de 1x1 muda de 1 canal pra 4:

- canais 0, 1, 2: logits de três classes, fundo / interior / fronteira entre instâncias, passados por softmax
- canal 3: mapa contínuo de distância ao fundo, passado por sigmoid porque o alvo é normalizado em [0,1] por instância

A normalização por instância do mapa de distância foi escolha nossa e vale defender: sem ela, o núcleo grande domina a regressão e o pequeno vira ruído numérico. O que a decodificação precisa é a forma do morro, onde fica o pico, não a altura absoluta em pixels.

Como gerar o rótulo de fronteira a partir das máscaras individuais

Essa é a primeira pergunta que o enunciado faz na trilha A, e a resposta é o detalhe que faz tudo funcionar. **A erosão é por instância, não na máscara semântica.** Para cada núcleo separadamente, o que sobra da erosão vira interior (classe 1) e a casca que a erosão comeu vira fronteira (classe 2).

Se a gente erodisse o foreground inteiro de uma vez, o contato entre dois núcleos encostados ficaria no meio de uma região de foreground e não viraria fronteira nenhuma, então o watershed não teria onde cortar e a trilha A degeneraria de volta na Parte 1. Erodindo instância a instância, no contato existe uma casca de cada lado, e a classe 2 aparece exatamente onde componentes conexos fundiria os dois. Tem um teste só pra isso, [test_boundary_separates_touching_instances](#), que constrói dois retângulos encostados sem um pixel de fundo entre eles e verifica que o interior sai com dois componentes.

O mapa de distância tem o mesmo cuidado: a EDT roda dentro do bounding box de cada instância, e não globalmente. Uma EDT global não teria vale nenhum no contato entre dois núcleos encostados, e o vale é

justamente o que o watershed usa pra decidir onde cortar. Também tem teste (`test_distance_does_not_leak_between_touching_instances`).

Que espessura

Segunda pergunta do enunciado. Espessura fina demais e a rede não consegue prever, uma casca de 1 px praticamente some no downsample de stride 32 do encoder. Grossa demais come o interior dos núcleos pequenos e eles deixam de virar marcador no watershed.

Medimos o trade-off em 200 imagens de treino e 9367 núcleos, com `uv run python scripts/class_stats.py --config configs/dsb2018_boundary.yaml` (saída em `results/class_stats.json`):

espessura	fundo	interior	fronteira	alpha (1/sqrt f)	marcadores perdidos	marcadores rachados
1	0.868	0.114	0.018	[0.28, 0.77, 1.95]	0	54
2	0.868	0.097	0.035	[0.33, 1.00, 1.67]	0	82
3	0.868	0.081	0.051	[0.36, 1.17, 1.47]	0	137
4	0.868	0.068	0.064	[0.36, 1.30, 1.34]	0	166

"Perdidos" é quantos núcleos ficam sem interior nenhum depois da erosão, ou seja, não viram marcador e viram falso negativo garantido. "Rachados" é quantos ficam com o interior partido em dois componentes, ou seja, viram dois objetos e produzem falso positivo garantido.

Ficamos com **espessura 2**. Nenhum núcleo perde o marcador em nenhuma das espessuras testadas (o código tem um cuidado específico pra isso: se a erosão apagaria o núcleo inteiro, ele guarda o pico da distância como interior). O que decide é a outra coluna: rachados cresce monotonicamente, de 54 pra 166, e espessura 2 é o ponto onde a fronteira já é grossa o bastante pra rede aprender sem estar rachando interior demais. Espessura 1 tem menos rachados mas deixa a classe 2 em 1.8% dos pixels, e nesse regime a rede simplesmente aprende a nunca prever fronteira.

Como pesar a classe fronteira, que é minoritária

Terceira pergunta. Com espessura 2 a fronteira é **3.5% dos pixels** e o fundo é 86.8%, uma razão de 25 para 1. É o desbalanceamento severo que o enunciado antecipa, e é exatamente o que o eixo 2 da Parte 3 vai medir.

Temos duas alavancas implementadas. A primeira é o peso por classe alpha da CE balanceada (slides 74 e 75), calculado de `alpha_from_frequencies`, que suporta $1/f$ e $1/\sqrt{f}$ com um teto. Usamos $1/\sqrt{f}$ porque $1/f$ puro coloca a fronteira em quase 30 vezes o peso do fundo e a loss vira praticamente só fronteira. A segunda é um mapa de peso por pixel (`touching_weight_map`), que dá peso extra só no contato entre duas instâncias diferentes, na linha da ideia do mapa de peso do paper original da U-Net. A distinção importa: a casca externa de um núcleo isolado não separa nada, só a casca entre dois encostados separa.

Por que isso resolve o problema da Parte 1

Em uma frase, que é como a gente vai falar na apresentação: componentes conexos só tem acesso à máscara binária, e dois núcleos encostados formam um blob único e conexo, então não há informação ali que permita separá-los. A trilha A faz a rede marcar explicitamente a casca de cada núcleo, e o interior que sobra já vem desconectado entre dois vizinhos. Componentes conexos sobre o interior já dá o número certo de objetos, e o watershed só precisa crescer cada marcador de volta até a borda real usando o mapa de distância como relevo.

Um detalhe de implementação que custou tempo: a máscara do watershed usa interior mais fronteira, não só interior. Se usasse só o interior, todos os núcleos saíam erodidos e o IoU com o ground truth nunca passaria de 0.5, o que zeraria o mAP inteiro mesmo com tudo mais perfeito. Tem teste pra isso também ([test_watershed_recovers_the_full_object_not_just_the_interior](#)). O relevo é **-dist** porque o watershed do skimage enche bacias a partir dos mínimos, então o centro do núcleo, onde a distância é máxima, precisa virar o fundo da bacia.

O resultado, lado a lado com a Parte 1

Mesmo encoder, mesmo decoder, mesmo split, mesmas 40 épocas, mesmo otimizador. Muda só a head de 1x1 (1 canal para 4), a perda e o pós-processamento. Split de teste, 101 imagens, matching guloso, em [results/parte2/metrics.json](#) e [results/parte2/comparacao/](#):

métrica	Parte 1 (limiar + CC)	Parte 2 (fronteira + watershed)	delta
IoU semântico	0.8415	0.8052	-0.0363
Dice	0.9120	0.8881	-0.0239
mAP @[.50:.95]	0.4654	0.4972	+0.0318
AP @.50	0.6931	0.7592	+0.0662
erro de contagem	10.05	4.10	-5.95

Esse é o slide principal da Parte 2, e o que ele mostra é melhor do que só "o mAP subiu". **As duas métricas se movem em direções opostas.** O Dice piorou, de 0.9120 para 0.8881, e o IoU semântico também. Pela métrica da aula, a Parte 2 é um modelo pior. E ao mesmo tempo o erro de contagem caiu **59%**, de 10.0 núcleos por imagem para 4.1, e o AP no limiar 0.50 subiu quase 7 pontos.

Dá pra explicar exatamente por que o Dice piora. A rede da Parte 2 gasta capacidade aprendendo uma casca de 2 px que representa 3.5% dos pixels e que, do ponto de vista semântico, é foreground igual ao interior. Quando ela erra pro lado de marcar fronteira demais, o foreground reconstruído (interior mais fronteira) fica um pouco mais magro que a verdade, e o Dice cai. Do ponto de vista de instância isso é um preço baixíssimo: perder 2 pontos de Dice pra cortar 59% do erro de contagem.

E o motivo de a apresentação insistir nisso é que o Dice é a métrica que a aula ensinou. Se a gente tivesse selecionado modelo por Dice, teria escolhido o modelo errado. Foi por isso que o loop de treino seleciona checkpoint por mAP de validação desde o começo.

Por limiar de IoU, o perfil dos dois:

limiar	0.50	0.55	0.60	0.65	0.70	0.75	0.80	0.85	0.90	0.95
Parte 1	0.693	0.663	0.633	0.601	0.572	0.509	0.423	0.314	0.188	0.058
Parte 2	0.759	0.727	0.697	0.659	0.614	0.552	0.461	0.328	0.157	0.018

Esses números são com os limiares de decodificação no chute, 0.5 pra tudo, que foi como a gente rodou primeiro. A seção seguinte mostra que isso estava deixando bastante desempenho na mesa nos **dois** modelos, e refaz a comparação direito.

A calibração da decodificação, e por que ela quase nos fez errar a análise

Os limiares dos dois pós-processamentos estavam no chute (0.5 pra tudo). Calibramos os dois **no split de validação** com `scripts/tune_watershed.py`, que roda a rede uma vez e varre os limiares em numpy, e só depois medimos no teste. Os melhores foram limiar 0.8 e min_size 20 pro baseline, e interior 0.5, foreground 0.7 e min_size 20 pra trilha A.

decodificação	Parte 1	Parte 2
padrão (limiar 0.5)	0.4654	0.4972
calibrada na validação	0.5351	0.5787

Os dois ganham muito, cerca de 0.07 e 0.08 de mAP, **sem tocar em um peso sequer da rede**. Os dois modelos preferem um limiar de foreground bem mais alto que 0.5, o que quer dizer que eles estão sistematicamente prevendo foreground demais, e cortar mais fundo melhora o IoU de cada instância.

Aqui a gente quase errou feio. Depois de calibrar só a Parte 2, ela passava a ganhar nos dez limiares de IoU, e a leitura óbvia era "o déficit da Parte 2 em IoU alto era artefato de hiperparâmetro". Só que essa comparação era injusta: um lado calibrado contra o outro no chute. Calibrando os dois, o padrão volta a aparecer, e ele é real. Fica de lição: calibrar só o método que a gente está defendendo é uma forma fácil de se enganar.

O resultado final, com os dois lados calibrados

Split de teste, 101 imagens, matching guloso, os dois com a decodificação escolhida na validação:

métrica	Parte 1 (limiar + CC)					Parte 2 (fronteira + watershed)					delta
IoU semântico	0.8415					0.8052					-0.0363
Dice	0.9120					0.8881					-0.0239
mAP @[.50:.95]	0.5351					0.5787					+0.0436
AP @.50	0.7312					0.8137					+0.0825
erro de contagem	8.57					4.07					-4.50
limiar	0.50	0.55	0.60	0.65	0.70	0.75	0.80	0.85	0.90	0.95	
Parte 1	0.731	0.701	0.677	0.650	0.618	0.579	0.524	0.442	0.314	0.114	
Parte 2	0.814	0.790	0.769	0.737	0.695	0.639	0.546	0.444	0.279	0.075	

O padrão é claro e sobrevive à calibração: **a Parte 2 ganha de 0.50 a 0.85 e perde em 0.90 e 0.95**. E a explicação original continua de pé. O watershed decide a divisa entre dois núcleos por um critério geométrico, a linha entre duas bacias do mapa de distância, e essa linha raramente coincide pixel a pixel com o traçado do anotador humano. Então a Parte 2 acerta muito mais objetos, cada um com contorno um pouco pior. Componentes conexos, quando por acaso acerta um núcleo isolado, acerta o contorno inteiro, porque ali o contorno é literalmente o limiar da probabilidade.

Como o mAP média dez limiares e sete deles estão na faixa onde a Parte 2 ganha, o saldo é fortemente positivo. E o erro de contagem, que é o que um biólogo realmente usaria, cai pela metade, de 8.57 pra 4.07 núcleos por imagem.

No sintético, onde praticamente todo objeto encosta em outro, o mesmo efeito aparece muito mais forte: mAP de 0.1032 para 0.5122 e erro de contagem de 9.70 para 2.22.

Parte 3, ablações nos eixos 2 e 3

Rodamos os eixos 2 (função de perda) e 3 (contexto global). Cada configuração com 2 seeds, reportando média e desvio, como o enunciado exige.

As ablações rodam com 20 épocas em vez das 40 do modelo final, e são 18 treinos ao todo (6 configurações no eixo 2 e 3 no eixo 3, cada uma com 2 seeds), 53 minutos de GPU.

Uma armadilha em que a gente caiu e vale contar. A ideia inicial era justificar o corte dizendo "na época 20 o modelo final já está em 0.4766 de mAP contra 0.4780 na época 40, então 20 épocas bastam". Isso está errado, e a gente só viu comparando os números depois. O scheduler é um CosineAnnealingLR com `T_max=epochs`, então mudar de 40 pra 20 épocas não corta o treino ao meio, muda a curva inteira de learning rate: com `T_max=20` a taxa já chegou perto de zero na época 20, enquanto com `T_max=40` ela ainda está na metade. Rodando a mesma configuração (focal balanceada gamma=2, seed 0) nos dois regimes:

época	4	8	12	16	20
dentro do schedule de 40 épocas	0.141	0.300	0.340	0.430	0.477
com schedule de 20 épocas	0.123	0.251	0.315	0.384	0.391

Ou seja, as ablações vivem num regime pior, e **os números absolutos delas não são comparáveis com os do modelo final**. O que continua valendo, e é o que importa aqui, é a comparação **dentro** de cada eixo: todas as configurações de um eixo compartilham o mesmo orçamento e o mesmo schedule, então a ordem entre elas é legítima. A gente prefere deixar isso escrito a fingir que não existe.

Deixamos o eixo 1 (como recuperar resolução) de fora por uma razão de método: pool indices, skip connections e atrous não são só três jeitos de fazer upsample, eles mudam a arquitetura junto. Comparar SegNet contra U-Net contra DeepLab mantendo tudo mais igual daria um experimento em que a variável isolada não é realmente uma variável só, e com o orçamento que a gente tinha preferimos dois eixos limpos a três sujus. Vale dizer que a análise de campo receptivo da Parte 5 acaba tocando no eixo 1 por outro caminho.

Eixo 2, a função de perda

O caminho que o enunciado desenha, CE, CE balanceada, focal, focal balanceada (slides 73 a 79), com gamma em {0, 1, 2, 5}. As seis configurações:

chave	perda	gamma	alpha
ce	CE	0	sem
ce_bal	CE balanceada	0	[0.45, 0.80, 1.75]
focal_g1	focal	1	sem
focal_g2	focal	2	sem
focal_g5	focal	5	sem
focal_g2_bal	focal balanceada	2	[0.45, 0.80, 1.75]

gamma = 0 é literalmente CE, o código é o mesmo (**FocalLoss** com gamma 0 reduz à CE), o que deixa o eixo contínuo e não um conjunto de perdas diferentes.

O que a gente espera antes de olhar, pra poder confrontar depois: com fundo em 86.8% dos pixels e fronteira em 3.5%, a CE pura deveria aprender a quase nunca prever fronteira, porque errar fronteira custa pouco na soma. Alpha ataca isso por classe e gamma ataca por dificuldade. A pergunta interessante é se as duas alavancas somam ou se uma anula a outra, que é o que a linha focal balanceada responde.

Eixo 3, contexto global

Os dois mecanismos dos slides 51 a 55, plugados no mesmo ponto da rede (topo do encoder, antes do decoder), variando só o módulo. Estão em [src/pa1/models/context.py](#).

Sem contexto é o braço de controle. ParseNet (image pooling, slides 51 e 52) tira a média global do feature map, projeta com 1x1, faz broadcast e concatena, então cada pixel passa a conhecer a estatística da imagem inteira. PSPNet (pyramid pooling, slides 53 a 55) faz pooling em várias grades (1x1, 2x2, 3x3, 6x6), projeta cada nível e concatena todos, o que dá contexto em várias escalas em vez de só global.

A pergunta específica do enunciado é boa e a gente montou a tabela pra responder ela diretamente:

contexto global ajuda a separar instâncias, ou só a classificá-las melhor? Por isso a tabela do eixo 3 reporta Dice (que mede classificar, ou seja, achar o objeto) do lado de mAP e erro de contagem (que medem separar). Se contexto só ajudasse a classificar, o Dice subiria e o mAP ficaria parado.

A intuição prévia, que a gente vai confrontar com o número: separar dois núcleos encostados é uma decisão sobre dois ou três pixels de contato, uma decisão local. Média global da imagem inteira não carrega informação sobre onde exatamente cortar. Então a expectativa é que contexto ajude pouco no mAP. Isso conversa com a análise da Parte 5.

Resultado do eixo 2

Split de validação, média e desvio sobre as seeds 0 e 1, em [results/parte3/eixo2_results.json](#):

configuração	mAP	Dice	erro de contagem
CE (gamma=0)	0.4879 +- 0.023	0.8929 +- 0.007	4.96 +- 0.11
CE balanceada	0.4189 +- 0.004	0.8613 +- 0.002	4.20 +- 0.34
focal gamma=1	0.4907 +- 0.020	0.8867 +- 0.010	5.00 +- 0.13

configuração	mAP	Dice	erro de contagem
focal gamma=2	0.4927 +- 0.000	0.8870 +- 0.006	4.59 +- 0.33
focal gamma=5	0.3965 +- 0.025	0.8392 +- 0.015	4.98 +- 0.06
focal balanceada gamma=2	0.3913 +- 0.012	0.8441 +- 0.001	4.40 +- 0.07

Três coisas, e a primeira contraria o que a gente tinha escrito antes de rodar.

Balancear piora, e não é pouco. CE cai de 0.4879 pra 0.4189 quando entra o alpha, e focal gamma=2 cai de 0.4927 pra 0.3913. São 0.07 e 0.10 de mAP, muito acima do desvio entre seeds. A gente tinha previsto o contrário, com o argumento de que a fronteira é 3.5% dos pixels e a CE pura ia ignorar ela.

A explicação que a gente defende, e que bate com o resto do trabalho: dar peso 1.75 pra fronteira faz a rede marcar fronteira **demais**, não apenas o suficiente. A casca prevista engorda, o interior mais fronteira reconstruído fica mais magro que o núcleo real, e o IoU de cada instância casada cai. A coluna do Dice confirma: ela cai junto (0.8929 para 0.8613 na CE, 0.8870 para 0.8441 na focal). Ou seja, o alpha não está corrigindo desbalanceamento, está desbalanceando pro outro lado.

O detalhe que fecha o argumento é o **erro de contagem indo na direção oposta**: com alpha ele melhora (4.96 para 4.20 na CE, 4.59 para 4.40 na focal). Faz sentido, porque fronteira mais grossa separa melhor. Então o alpha faz exatamente o que se esperava dele, separar melhor, e o preço em delineamento é maior que o ganho. É o mesmo trade-off entre separar e delinear que apareceu na Parte 2 e que vai reaparecer na correção da Parte 5. Se a métrica fosse só contagem de células, que é o que um biólogo normalmente quer, a escolha seria a oposta.

gamma quase não importa entre 0 e 2. 0.4879, 0.4907 e 0.4927 para gamma 0, 1 e 2, com desvio de até 0.023. As três empatam. Só gamma=5 quebra (0.3965), e aí a explicação é a usual: com gamma tão alto quase todo pixel vira "fácil" e o gradiente some.

O desvio entre seeds é pequeno, no máximo 0.025, o que dá confiança de que as diferenças de 0.07 e 0.10 acima são reais e não ruído.

Resultado do eixo 3

Mesmo protocolo, variando só o módulo de contexto no topo do encoder. Em

[results/parte3/eixo3_results.json](#):

configuração	mAP	Dice	erro de contagem
sem contexto	0.3895 +- 0.012	0.8444 +- 0.007	4.45 +- 0.27
image pooling (ParseNet)	0.4253 +- 0.019	0.8584 +- 0.012	4.53 +- 0.20
pyramid pooling (PSPNet)	0.4097 +- 0.024	0.8532 +- 0.015	3.96 +- 0.19

Antes de interpretar, uma ressalva de método: o eixo 3 roda em cima da perda do config original, que é a focal balanceada, e o eixo 2 mostrou depois que ela é uma das piores. Por isso o braço "sem contexto" aqui está em 0.3895 e não perto de 0.49. As três barras compartilham essa perda, então a comparação entre elas continua valendo, mas o patamar todo está rebaixado.

A resposta pra pergunta do enunciado, que era se contexto global ajuda a separar instâncias ou só a classificá-las melhor: **majoritariamente a classificar**.

Os dois módulos sobem o Dice de forma consistente, +0.014 no ParseNet e +0.009 no PSPNet. Ou seja, os dois ajudam a decidir se um pixel é núcleo ou fundo, que é classificação. Mas no erro de contagem, que é a medida direta de separação, o ParseNet **piora** (4.45 para 4.53) e só o PSPNet melhora (4.45 para 3.96, 11%).

Isso conversa exatamente com a análise de campo receptivo da Parte 5. Média global da imagem inteira, que é o que o ParseNet faz, é um único vetor por imagem: ele diz "isso aqui é uma lâmina de fluorescência escura" e ajuda a calibrar o limiar de foreground, mas não carrega nenhuma informação sobre **onde** cortar entre dois núcleos vizinhos, porque essa é uma decisão sobre dois ou três pixels de contato. Já o PSPNet faz pooling em grades 2x2, 3x3 e 6x6 além da global, e essas grades ainda têm alguma localização, o que explica ele ser o único que mexe no erro de contagem.

Resumindo pra apresentação: contexto global melhora o mAP, mas por classificar melhor, não por separar melhor. Quem separa melhor é o contexto **multi-escala**, e mesmo assim modestamente.

O que a Parte 3 mudou no modelo final, e por que quase nada mudou

As ablações não ficaram como apêndice: a gente levou as conclusões delas de volta pro modelo final e mediu. Duas mudanças, testadas em `configs/dsb2018_final.yaml` (sem alpha) e `configs/dsb2018_final_ctx.yaml` (sem alpha mais image pooling), as duas treinadas com as 40 épocas do modelo final e as duas com a decodificação calibrada na própria validação.

O critério de escolha é o mAP de validação, cada modelo no seu próprio ótimo de decodificação:

modelo	perda	contexto	mAP de validação
<code>dsb2018_boundary</code> (o original)	focal gamma=2 com alpha	nenhum	0.5612
<code>dsb2018_final</code>	focal gamma=2 sem alpha	nenhum	0.5344
<code>dsb2018_final_ctx</code>	focal gamma=2 sem alpha	image pooling	0.5539

Tirar o alpha não ajudou no orçamento cheio, apesar de ter ajudado no orçamento das ablações. No regime de 20 épocas a diferença entre com e sem alpha era de 0.10 de mAP a favor de tirar. Com 40 épocas e o schedule completo, a diferença inverte e fica em 0.027 a favor de manter.

Esse é um resultado negativo e a gente vai apresentar ele como tal, porque ele é mais informativo que o positivo teria sido. A leitura: **a conclusão da ablação não transferiu para o regime do modelo final**. É consistente com a armadilha do scheduler que a gente já tinha detectado. O alpha empurra a rede a marcar fronteira demais, o que atrapalha cedo no treino, mas com learning rate alto por mais tempo a rede tem folga pra desfazer esse viés e acaba aproveitando o sinal extra na classe minoritária. Ou seja, o alpha não é ruim, ele é **lento**, e o orçamento de 20 épocas não dava tempo de ele pagar.

O que isso custa: não dá pra usar uma ablação barata como substituta de um experimento no regime real, mesmo que ela seja limpa, com 2 seeds e desvio pequeno. O ranking que ela produz vale dentro do orçamento dela, e só.

O image pooling ficou no meio (0.5539) e também não superou o original, mas ele reproduz de novo o achado do eixo 3 de forma bem nítida: é o modelo com **melhor Dice de todos** (0.9139 no teste, contra

0.8881 do original) e ao mesmo tempo com um dos piores mAP. Contexto global melhora classificação e não melhora separação, medido duas vezes em regimes diferentes.

O modelo final entregue continua sendo o `dsb2018_boundary`, com a decodificação calibrada. As mudanças testadas ficaram no repo com o número delas, que é o que dá lastro pra afirmar que foram testadas e não apenas cogitadas.

Parte 4, inferência em mosaico

O slide 83 descreve a prática padrão pra imagem grande: processa em tiles com patches sobrepostos, considera a parte interna e faz a média dos resultados. Isso resolve segmentação semântica.

Por que quebra pra instância. O id de uma instância é arbitrário. O núcleo 7 do tile da esquerda e o núcleo 3 do tile da direita podem ser o mesmo núcleo, e não existe média entre 7 e 3. Média de rótulo não é uma operação definida. Pior, um núcleo cortado pela emenda vira dois objetos, cada um com aproximadamente metade da área, e nenhum dos dois casa com o ground truth nem no limiar mais frouxo de IoU 0.50.

Implementamos quatro estratégias (`src/pa1/tiling.py`, rodadas por `scripts/part4_mosaic.py`):

- **full**: sem tiling, imagem inteira de uma vez. É o teto de referência.
- **per_tile**: decodifica dentro de cada tile e cola só a parte interna, que é o slide 83 aplicado ao pé da letra. É o que a gente mostra quebrando.
- **blend**: faz a média dos logits na sobreposição e decodifica **uma vez só** no fim, na imagem inteira.
- **fuse**: decodifica por tile e depois costura, unindo instâncias de tiles vizinhos que se sobrepõem na faixa comum.

A correção que o item 4 pede é a **fuse**. O critério é IoU na faixa de sobreposição, com limiar 0.25, e a união é transitiva via union-find. As duas decisões têm motivo. IoU e não "encostou" porque dois núcleos vizinhos de verdade também encostam e não podem ser fundidos, e o que separa os casos é que instâncias correspondentes têm IoU alto na faixa comum enquanto vizinhos legítimos têm IoU perto de zero. Union-find porque um núcleo que aparece em três tiles precisa virar um objeto e não dois, e união par a par sem transitividade deixaria isso passar. Tem teste pros três casos em `tests/test_tiling.py`: objeto na emenda vira dois pedaços no `per_tile` e um só no `fuse`, dois objetos genuinamente distintos continuam dois, e objeto atravessando três tiles sai como um.

A **blend** merece um comentário conceitual que vale na apresentação: ela é a que mais se aproxima do espírito do slide 83, e o único motivo de ela ser possível é que a nossa representação é densa. Dá pra fazer média de logit de fronteira porque logit é um número comparável entre rodadas. Um detector com proposta de região, que é o que o PA proibiu, não teria esse caminho: não existe média entre duas caixas propostas em tiles diferentes, só NMS, que é justamente uma forma de fusão como a nossa. Ou seja, a proibição do enunciado nos empurrou pra representação que torna o problema do tiling mais fácil, não mais difícil.

O resultado

Mosaico de 3x3 imagens do teste (as 9 mais densas), 768x768, com 419 instâncias no ground truth. Rodamos com três tamanhos de tile de propósito, porque tile menor cria mais emenda: com tile 256 são 9 tiles, com tile 96 são 81. Em `results/parte4/`.

tile / sobreposição	full	per_tile	blend	fuse
256 / 64	0.3512	0.2741	0.3527	0.3374
128 / 32	0.3512	0.2061	0.3476	0.3148
96 / 16	0.3512	0.1751	0.3318	0.2856

E a contagem de objetos, que é onde a falha fica gritante (o ground truth tem 419):

tile / sobreposição	full	per_tile	blend	fuse
256 / 64	360	461	361	356
128 / 32	360	536	360	348
96 / 16	360	621	356	337

Essa segunda tabela é o slide. Sem tiling o modelo prevê 360 objetos, já subestimando. Com o método do slide 83 aplicado ao pé da letra e tile 96, ele prevê **621**, um excesso de 48% sobre o ground truth. O modelo não mudou, a rede é a mesma, os pesos são os mesmos. Os 261 objetos a mais foram **criados pelo pós-processamento**, são núcleos cortados pelas emendas e contados duas vezes. E a degradação é monotônica: quanto menor o tile, mais emenda, mais objeto fantasma e menos mAP (0.274, 0.206, 0.175).

O **blend** é praticamente imune (0.353, 0.348, 0.332, contagem sempre perto de 360) e o **fuse** recupera boa parte (0.337, 0.315, 0.286).

Olhando só nas instâncias que efetivamente cruzam uma emenda:

tile	instâncias na emenda	IoU médio, per_tile	IoU médio, fuse	recuperadas em 0.50
256	7	0.516	0.606	5 e 5
128	11	0.543	0.521	5 e 6
96	14	0.597	0.657	10 e 12

A costura melhora o IoU médio dessas instâncias em dois dos três casos. No tile 128 ela piora um pouco, e a explicação é que a fusão por IoU às vezes une dois núcleos vizinhos de verdade que aparecem juntos em dois tiles, o que troca dois objetos certos por um errado. É o preço de decidir primeiro e consertar depois.

Por que blend ganha de fuse. Blend faz a média antes de decidir, então o watershed roda uma vez só sobre um mapa contínuo consistente na imagem inteira, e o problema de identidade de instância nem chega a existir. Fuse decide primeiro e conserta depois, e conserto depois de uma decisão errada nunca recupera tudo: se o watershed já cortou um núcleo ao meio dentro de um tile, unir os dois pedaços devolve a área mas não devolve o contorno que teria saído de uma decisão única.

A conclusão que vale pra apresentação é que o slide 83 está certo, mas o "faça a média dos resultados" precisa ser lido como **média da saída densa da rede, antes de decodificar**, e não média do resultado final. Pra segmentação semântica os dois são a mesma coisa, porque não existe passo de decodificação. Pra instância são coisas completamente diferentes.

E vale notar a ironia: o único motivo de o **blend** existir é que a nossa representação é densa. Um detector com proposta de região, que é o que o PA proibiu, não teria esse caminho, porque não existe média entre duas caixas propostas em tiles diferentes, só NMS, que é justamente uma forma de fusão como a nossa e sofre do mesmo problema. Ou seja, a proibição do enunciado nos empurrou pra representação que torna o problema do tiling **mais fácil**, não mais difícil.

Parte 5, campo receptivo teórico e galeria de falhas

O campo receptivo (item obrigatório)

A conta é a recorrência padrão dos slides 35 a 38, implementada em `src/pa1/receptive_field.py`. Percorrendo as camadas em ordem, com `j` o espaçamento acumulado e `r` o campo receptivo:

```
j_out = j_in * s
r_out = r_in + (k - 1) * d * j_in
```

com `k` o tamanho do kernel, `s` o stride e `d` a dilatação. Começa em `j = 1` e `r = 1`. O slide 38 é exatamente isso: pooling aumenta `r` sem custar parâmetro, mas cobra em resolução. O slide 39 mostra a alternativa atrous, que aumenta `r` sem mexer em `j` nem no número de pesos.

Para o encoder do modelo final, ResNet34, por estágio:

estágio	campo receptivo	stride acumulado
stem (conv 7x7 + maxpool)	11 px	4
fim do layer1	59 px	4
fim do layer2	179 px	8
fim do layer3	547 px	16
fim do layer4	899 px	32

E comparando encoders e a variante atrous, todos na mesma conta:

encoder	campo receptivo	output stride	parâmetros
ResNet34 (o nosso)	899 px	32	24.44M
ResNet34, atrous no layer4	931 px	16	24.44M
ResNet34, atrous no layer3 e layer4	947 px	8	24.44M
ResNet18	435 px	32	
ResNet18, atrous no layer4	467 px	16	

A coluna de parâmetros é o argumento do slide 39 verificado na prática: dilatar não adiciona um peso sequer, o filtro é o mesmo com buraco no meio. O que muda é só a resolução do mapa de saída. Uma pegadinha de implementação: o **BasicBlock** do torchvision (que é o bloco da ResNet18 e da ResNet34) recusa

`replace_stride_with_dilation`, só o `Bottleneck` aceita, então a gente aplicou a dilatação nas convs na mão em `src/pa1/models/encoders.py`.

A comparação com o tamanho dos objetos, e a surpresa

Os núcleos do DSB2018 têm diâmetro equivalente mediano de **19.4 px** no split de teste (4371 instâncias), p95 de 47.8 px e máximo de 93.4 px. No treino (9367 instâncias) a mediana é 20.7 px, a média 21.9 px e o máximo 87.7 px. Ou seja, os dois splits contam a mesma história, e o histograma está em `results/parte5/receptive_field.png`.

Confrontando com a tabela acima, o diagnóstico que o próprio enunciado dá como exemplo ("o objeto tem 180 px de diâmetro e o campo receptivo teórico do meu encoder é 140 px") **não se aplica ao nosso caso, e por uma margem enorme**. O campo receptivo teórico do ResNet34 é 899 px e o maior núcleo do dataset tem 93 px. Nem o maior objeto chega a um décimo do campo receptivo. Se a nossa única ferramenta de diagnóstico fosse campo receptivo, a conclusão seria que não há nada errado, e claramente há.

A coluna que conta a história certa é a outra: **output stride 32**. O feature map mais profundo tem uma célula a cada 32 px da imagem, e o núcleo mediano tem 19.4 px de diâmetro. Ou seja, **um núcleo inteiro é menor do que uma única célula do topo do encoder**, e dois núcleos encostados cabem folgados dentro da mesma célula. No fim do layer4 não existe representação nenhuma capaz de distinguir "um núcleo" de "dois núcleos encostados", porque os dois casos produzem exatamente a mesma ativação naquela resolução.

Isso reposiciona todo o resto do trabalho. A informação que separa as instâncias só pode vir dos estágios rasos, onde o stride ainda é 4 ou 8, e chega ao decoder pelas skip connections, não pelo caminho profundo. E é por isso que a nossa previsão pro eixo 3 é de ganho pequeno em mAP: contexto global se pluga no topo do encoder, exatamente na resolução onde a informação de separação já foi destruída. Contexto global pode ajudar a decidir se aquilo é núcleo ou não (classificar), mas não tem como ajudar a decidir onde cortar entre dois (separar).

Isso também responde à pergunta do enunciado sobre atrous. Como o campo receptivo já é grande demais, o valor de atrous aqui **não é o campo receptivo, é o output stride**: dilatar o layer4 leva o stride de 32 pra 16, e dilatar layer3 e layer4 leva pra 8, que já é menor que o diâmetro de um núcleo. O ganho de campo receptivo que vem junto (899 para 931 para 947 no ResNet34) é irrelevante nesse dataset, porque 899 já era grande demais. Essa é a leitura que a gente quer defender na apresentação: o mesmo mecanismo do slide 40, mas o benefício dele aqui é o efeito colateral, não o efeito anunciado.

A galeria de falhas

As cinco piores imagens do teste estão em `results/parte5/failure_1.png` a `failure_5.png`, cada uma com imagem, ground truth, predição e os quatro mapas intermediários (foreground, interior, fronteira e distância), que é o que o enunciado pede. Os números que sustentam o diagnóstico de cada uma:

#	imagem	AP	gt	previstas	fundidas	fragmentadas	diâmetro mediano
1	942d56861f	0.037	51	54	0	0	16 px
2	3a3fee427e	0.091	56	56	2	2	18 px
3	13c8ff1f49	0.112	17	14	2	0	13 px
4	ad473063da	0.113	84	60	18	0	12 px

#	imagem	AP	gt	previstas	fundidas	fragmentadas	diâmetro mediano
5	358e47eaa1	0.134	52	54	2	2	19 px

O padrão salta aos olhos: **as cinco têm núcleo pequeno**, de 12 a 19 px de diâmetro mediano, contra 19.4 px da mediana do dataset, e quatro das cinco são imagens densas, com 51 a 84 núcleos. O caso 1 é o mais instrutivo, porque ele não tem nenhuma fusão nem fragmentação (a contagem quase bate, 54 contra 51) e mesmo assim tira AP 0.037. Ou seja, ele achou quase o número certo de objetos e errou o **contorno** de praticamente todos. Isso é falha de delineamento, não de separação, e é um modo de erro diferente do que a Parte 1 tinha.

O caso 4 é o oposto e é o modo de falha clássico: 84 núcleos verdadeiros, 60 previstos, 18 instâncias previstas cobrindo dois ou mais núcleos de verdade. É o aglomerado denso onde a casca de 2 px entre núcleos simplesmente não foi prevista.

O diagnóstico, com o número do erro no split inteiro

Somando os quatro modos de erro nas 101 imagens de teste, nos dois modelos

([results/parte5/results.json](#) pra trilha A e [results/parte5/baseline_results.json](#) pro baseline):

modo de erro	Parte 1 (limiar + CC)	Parte 2 (fronteira + watershed)
fusões (uma previsão cobre 2+ núcleos)	582	260
fragmentações (2+ previsões num núcleo)	4	67
núcleos não achados	1505	886
previsões espúrias	510	550

Essa tabela é a versão quantitativa do que a Parte 2 prometeu. A trilha A **cortou as fusões em 55%**, de 582 pra 260, que era exatamente o objetivo. E o preço está na linha de baixo: a fragmentação, que praticamente não existia na Parte 1 (4 casos), subiu pra 67. É o modo de falha novo que a representação introduz, quando o interior previsto racha em dois marcadores e o watershed corta um núcleo saudável ao meio. Trocamos 322 fusões por 63 fragmentações, o que é um bom negócio, mas não é de graça.

A correção, e o que ela revelou

O diagnóstico da seção anterior diz que o problema não é campo receptivo, é resolução: com output stride 32 um núcleo de 19 px cabe dentro de uma célula do mapa mais profundo. A mudança que isso sugere é direta e é o mecanismo do slide 40: dilatar o layer4 pra levar o output stride de 32 pra 16, sem adicionar parâmetro nenhum. Está em [configs/dsb2018_boundary_os16.yaml](#), e o antes/depois no mesmo split de teste:

métrica	output stride 32	output stride 16	mudou
mAP @[.50:.95]	0.4972	0.4830	pior
AP @.50	0.7592	0.7742	melhor
AP @.90	0.157	0.121	pior
Dice	0.8881	0.8857	igual

métrica	output stride 32	output stride 16	mudou
erro de contagem	4.10	3.77	melhor
fusões	260	251	melhor
núcleos não achados	886	798	melhor
fragmentações	67	71	pior

A correção funcionou no que o diagnóstico previa e falhou no total. Todas as medidas de separação melhoraram: AP no limiar frouxo subiu 1.5 ponto, o erro de contagem caiu 8%, as fusões caíram e os núcleos não achados caíram 10%. Mas o mAP agregado caiu 0.014, porque os limiares severos (0.90 e 0.95) pioraram.

O que isso revela, e é a parte interessante: o diagnóstico estava **certo sobre o mecanismo e incompleto sobre a consequência**. Ele previa que mais resolução no topo do encoder ajudaria a separar, e ajudou. O que ele não considerou é que dilatar o layer4 introduz o efeito de gridding do atrous (o filtro passa a amostrar pixels alternados, e pixels vizinhos passam a ser processados por conjuntos disjuntos de pesos), o que degrada o contorno fino. Separação melhora, delineamento piora, e como o mAP média dez limiares de IoU, sendo metade deles severos, o delineamento domina o agregado.

Vale notar que esse é o **terceiro** lugar do trabalho onde aparece o mesmo trade-off entre separar e delinear: a Parte 2 contra a Parte 1, o alpha do eixo 2, e agora o output stride. Nos três casos a mesma tensão, e nos três a métrica agregada esconde o que está acontecendo. Se o objetivo fosse contar células, as três decisões seriam tomadas na direção oposta.

Parte 6, teste de estresse por corrupção

Escolhemos a opção das corrupções, entre as três oferecidas, porque é a única que produz uma curva de degradação de verdade, com eixo x ordenado, em vez de dois pontos soltos. São três famílias em três intensidades cada ([src/pa1/corruptions.py](#)):

corrupção	intensidade 1	2	3
blur gaussiano	sigma 1.0	2.0	4.0
ruído gaussiano	desvio 8	20	40
brilho e contraste	ganho 0.75, viés -10	0.55, -25	0.35, -40

As corrupções entram só na avaliação, nunca no treino, tirando o brilho e contraste leve que já está no augment. Essa assimetria é o ponto do teste.

A tabela reporta Dice do lado do mAP de propósito, pela mesma lógica do eixo 3: se o Dice cai junto, o modelo está perdendo a capacidade de achar o objeto, e se o Dice segura e só o mAP cai, ele ainda vê os núcleos mas perdeu a capacidade de separá-los, o que localizaria o dano na cabeça de fronteira e não no reconhecimento.

A previsão a confrontar: blur deveria ser o pior dos três, porque a classe fronteira é uma casca de 2 px e um blur de sigma 4 destrói literalmente a estrutura que a rede precisa prever. Ruído e brilho não atacam a geometria, só o contraste.

O resultado

Split de teste inteiro, modelo da trilha A. Em [results/parte6/results.json](#) e a curva em [results/parte6/degradation.png](#):

corrupção	intensidade	mAP	Dice	erro de contagem
nenhuma	0	0.4972	0.8881	4.10
ruído	1	0.5145	0.8990	4.37
ruído	2	0.4328	0.8725	4.93
ruído	3	0.3286	0.8312	5.75
blur	1	0.4265	0.8582	4.16
blur	2	0.2594	0.7536	5.13
blur	3	0.1731	0.6346	8.35
brilho e contraste	1	0.4452	0.8701	4.39
brilho e contraste	2	0.2412	0.6692	10.15
brilho e contraste	3	0.0931	0.3499	24.06

Três coisas, e as duas primeiras contrariam o que a gente tinha previsto.

Ruído leve melhora o modelo. Com intensidade 1 o mAP sobe de 0.4972 pra 0.5145 e o Dice de 0.8881 pra 0.8990. Não é ruído de medição, são 101 imagens e o efeito aparece nas duas métricas. A explicação é que [A.GaussNoise](#) está no augment de treino, então ruído leve e dentro da distribuição que a rede viu, e adicionar um pouco funciona como leve regularização na inferência. É um lembrete útil: "corrupção" não é sinônimo de "pior", o que importa é a distância pra distribuição de treino, não a degradação perceptual.

Blur não é o pior, brilho e contraste é. A gente tinha previsto blur como o pior, porque a classe fronteira é uma casca de 2 px e um blur de sigma 4 destrói a estrutura que a rede precisa prever. Blur 3 de fato derruba pra 0.1731, mas brilho e contraste 3 derruba pra **0.0931**, quase o dobro de dano. O motivo aparece na coluna do Dice: com brilho 3 o Dice desaba pra 0.3499, ou seja, o modelo perde o objeto inteiro, não só a separação. Ganho 0.35 com viés -40 achata a imagem quase toda numa faixa estreita de cinza escuro, e o encoder pré-treinado na ImageNet nunca viu nada assim. O augment de treino tem brilho e contraste, mas com limite 0.25, muito mais suave que os 0.65 da intensidade 3.

A terceira leitura é a que responde à pergunta de projeto, e sai de olhar mAP e Dice juntos, que é o motivo de a tabela ter as duas colunas:

corrupção intensidade 3	queda do Dice	queda do mAP	razão
ruído	-6%	-34%	5.6x
blur	-29%	-65%	2.3x
brilho e contraste	-61%	-81%	1.3x

Com ruído o Dice quase não se mexe (cai 6%) e o mAP cai 34%. Ou seja, sob ruído o modelo **continua enxergando os núcleos e perde a capacidade de separá-los**. Isso localiza a fragilidade exatamente onde a gente esperava: na cabeça de fronteira, que precisa acertar uma casca de 2 px e é por construção a parte mais fina e mais sensível da representação. Com brilho e contraste a razão vai pra 1.3, ou seja, ali o dano é generalizado, o modelo perde tudo junto.

O custo prático dessa fragilidade está na última coluna da tabela grande: com brilho e contraste 3 o erro de contagem vai de 4 pra **24 núcleos por imagem**, que é mais do que a Parte 1 errava na imagem limpa.

O que ficou de fora, e o que a gente faria com mais tempo

Vale ser explícito sobre os limites do que está aqui, porque na apresentação alguém vai perguntar.

O eixo 1 da Parte 3 não foi rodado. Escolhemos os eixos 2 e 3, que é o que o enunciado pede (dois dos três). A comparação entre pool indices, skip connections e atrous ficou de fora como experimento controlado, embora a análise de campo receptivo da Parte 5 e a correção com output stride 16 toquem no mesmo assunto por outro caminho.

A busca de hiperparâmetro foi mínima. Learning rate, weight decay, tamanho do crop e número de épocas foram escolhidos de uma vez e não variados. O único ajuste feito com método foi a decodificação do watershed, varrida na validação com `scripts/tune_watershed.py`, e a espessura da fronteira, escolhida pela tabela de marcadores perdidos e rachados. Tudo mais é chute informado.

Uma única seed no modelo final. As 2 seeds exigidas estão nas ablações da Parte 3. O modelo final da Parte 2 rodou com seed 0 só, então a diferença de 0.0436 de mAP entre Parte 1 e Parte 2 não vem com barra de erro. O que dá confiança de que o efeito é real é que ele aparece com o mesmo sinal e muito maior no dataset sintético, e que o erro de contagem cai 53%, que é uma diferença grande demais pra ser ruído de seed.

O teste foi olhado mais de uma vez. A gente selecionou checkpoint pela validação e calibrou o watershed pela validação, o que está certo, mas rodou a avaliação de teste várias vezes ao longo do desenvolvimento. Não houve escolha de modelo feita pelo número de teste, mas registrar isso é mais honesto do que fingir que o teste foi aberto uma vez.

As ablações rodaram num regime que não é o do modelo final, e a gente só descobriu o tamanho disso no fim, quando levou a conclusão do eixo 2 de volta pro orçamento cheio e ela não se sustentou. O certo teria sido rodar as ablações com 40 épocas, o que custaria umas 2 horas de GPU em vez de 53 minutos, ou pelo menos fixar o schedule em vez de deixar o T_max seguir o número de épocas. Ficou como está, documentado, porque o resultado negativo também ensina.

Com mais tempo, na ordem em que a gente atacaria: refazer o eixo 2 com 40 épocas pra ver se o ranking muda mesmo ou se foi só o schedule, rodar o modelo final com 3 seeds pra ter barra de erro na comparação principal, tentar a trilha B pra ver se embedding separa melhor que fronteira nos aglomerados densos do cluster 3, e treinar com output stride 8 pra levar o diagnóstico da Parte 5 até o fim.

Mapa dos entregáveis

item do enunciado	onde está
repositório com histórico	commits ao longo do desenvolvimento, não um só

item do enunciado	onde está
README com ambiente, dados, um comando que treina, um que avalia	README.md
AI_LOG	AI_LOG.md
notebook de inferência que roda sem retreinar	inferencia.ipynb , lógica em src/pa1/inference.py
checkpoint do modelo final	runs/dsb2018_boundary/best.pt , link no README
Parte 3 aplicada de volta no modelo final	configs/dsb2018_final*.yaml , resultado negativo documentado
Parte 0, gerador sintético e teste em menos de 5 min	src/pa1/data/synthetic.py , configs/synthetic*.yaml
Parte 1, baseline binário, IoU e Dice	configs/dsb2018_baseline.yaml , results/parte1/
Parte 1, instâncias por limiar e componentes conexos	src/pa1/postprocess/naive.py
Parte 1, mAP com matching próprio	src/pa1/metrics/instance.py , tests/test_instance_metrics.py
Parte 1, regra de matching documentada	seção da métrica aqui, e as duas regras medidas
Parte 1, gráfico contra densidade	results/parte1/density.png
Parte 2, trilha A	src/pa1/data/targets.py , src/pa1/postprocess/watershed.py
Parte 2, métricas lado a lado	scripts/compare.py , results/parte2/comparacao/
Parte 3, eixos 2 e 3, 2 seeds	scripts/ablations.py , results/parte3/
Parte 4, mosaico, falha e correção	src/pa1/tiling.py , results/parte4/
Parte 5, campo receptivo e galeria	src/pa1/receptive_field.py , results/parte5/
Parte 5, a correção	configs/dsb2018_boundary_os16.yaml
Parte 6, corrupções	src/pa1/corruptions.py , results/parte6/

Como reproduzir tudo

Os comandos estão no README, na seção "Reproduzir cada parte". Em GPU o pipeline inteiro das Partes 0, 1, 2, 4, 5 e 6 leva em torno de 30 minutos, e a Parte 3 leva algumas horas porque são 18 treinos. Em CPU tudo roda igual, só muito mais devagar.

Nenhuma das nossas máquinas tem GPU NVIDIA, então os treinos saíram de kernel do Kaggle com o notebook de [notebooks/colab_setup.ipynb](#) adaptado. Uma pegadinha que custou tempo: o Kaggle às vezes entrega uma P100, que é sm_60, e o PyTorch da imagem deles só cobre sm_70 pra cima, então [torch.cuda.is_available\(\)](#) dá True e nada roda. O notebook checa [torch.cuda.get_device_capability\(\)](#) no início e instala um torch compatível se precisar.